

RootPulse

Product Requirements Document

Uptime, Incident & Dependency Monitoring Platform

Free-tier MVP — single-user, zero ongoing cost

Version: 1.2 (Draft)

Date: 2026-09-28

Status: Draft — for review

Owner: Product (you)

Prepared with: Claude (research basis: public UptimeRobot documentation plus user-provided screenshots of their own account, see Appendix A)

v1.2 changes: product name confirmed as **RootPulse** (was a placeholder). **v1.1 changes:** incorporated UX patterns confirmed from the user's own authenticated UptimeRobot account (Free plan), screenshotted 2026-09-28 — see Section 6.11 and updated notes in Sections 6.1, 9.1, and Appendix A.

Table of Contents

1. Executive Summary
2. Goals & Success Metrics
3. Target Users & Personas
4. Problem Statement & Value Proposition
5. Scope — MVP Boundaries
6. Functional Requirements
 - 6.1 Core Monitoring
 - 6.2 Alerts & Notifications
 - 6.3 Incident Management
 - 6.4 Status Pages
 - 6.5 Analytics Dashboard
 - 6.6 Dependency Monitoring (Small Free Version)
 - 6.7 Deployment-Aware Incidents
 - 6.8 Root-Cause Timing Breakdown
 - 6.9 Reports & Export
 - 6.10 Account, Auth & API
 - 6.11 Onboarding & UX Patterns
7. Non-Functional Requirements
8. Suggested Free-Tier Architecture
9. Key User Flows
10. Data Model Overview
11. Out of Scope for MVP (Deferred to V2 / Future)
12. Risks & Assumptions
13. Roadmap
14. Open Questions
- Appendix A — Competitive Research Basis

1. Executive Summary

This document specifies the requirements for **RootPulse**, a self-built, zero-cost uptime and incident monitoring platform, positioned as a free MVP inspired by (but not a clone of) UptimeRobot. The product checks websites, APIs, servers, and scheduled jobs for availability; alerts the owner through free notification channels when something breaks; records and helps resolve incidents; communicates status through a public status page; and reports on reliability trends through a built-in analytics dashboard.

The defining constraint of this MVP is **cost: \$0 ongoing spend**. Every feature in this document was selected or scoped specifically to be buildable on free hosting tiers, free notification channels, and open-source tooling — with no paid APIs, no SMS/voice credits, and no paid third-party datasets. Two capabilities that would normally require a paid LLM API (deployment-correlated incidents and root-cause timing breakdowns) are included because they can be built as pure data correlation and instrumentation, with no AI/LLM cost at all.

The product is single-user for this phase: no team roles, seats, or billing system are in scope. That infrastructure is deferred until there is a reason to charge for the product.

2. Goals & Success Metrics

2.1 Product goals

- 1 Detect downtime and other failures (SSL/domain expiry, DNS changes, missed cron jobs) reliably, with a low false-positive rate.
- 2 Notify the owner within seconds of a confirmed failure, through channels that cost nothing to run.
- 3 Make it fast to understand *why* something broke, not just that it broke, using data the system already has (recent deploys, request timing) — without calling a paid AI service.
- 4 Give a single place to communicate status publicly and see reliability trends over time.
- 5 Prove the free-tier architecture can scale to a small number of real monitors (dozens, not thousands) before any infrastructure spend is considered.

2.2 Success metrics (MVP)

Metric	Target	Why it matters
Mean detection time	< 2 minutes from failure to confirmed incident	Core promise of the product
False-positive rate	< 5% of opened incidents	Multi-location re-checks exist specifically to hold this down
Alert delivery success	> 99% of alerts delivered on a free channel	A monitor that can't notify you is useless
Monthly infra cost	\$0	Hard constraint for this phase
Status page load time	< 1.5s	Public-facing; reflects on the product it's monitoring

3. Target Users & Personas

The MVP targets a single primary persona; broader personas from the differentiation research (agencies, small eng teams) become relevant once multi-user support is built.

Primary persona: the solo builder / site owner

- Runs one or a handful of websites, APIs, or side projects.
 - Wants to know immediately if something goes down — currently either has no monitoring or uses a free tier of an existing tool.
 - Not willing or able to pay for monitoring right now.
 - Comfortable with Slack/Telegram/Discord/email; does not need SMS or voice calls.
 - Wants a public status page to look credible to visitors or the few people they answer to.
-

4. Problem Statement & Value Proposition

Problem: “Is my thing up, and will I know before someone else tells me?” Free monitoring tools answer the first half well. They rarely explain *why* something broke, and rarely connect an incident to a recent deploy — the single most common real-world cause of an outage for a small project.

Value proposition: a free monitoring tool that still tells you, in plain data, whether an incident is your own deploy, and which stage of the request failed (DNS, TCP, TLS, or the server itself) — without needing a paid AI subscription or an enterprise plan to see it.

5. Scope — MVP Boundaries

Hard constraint: \$0 ongoing cost. Every feature below must run on free hosting/service tiers. Where a feature would normally require a paid service (SMS/voice, a licensed provider-incident dataset, an LLM API), it is either rescoped to a free equivalent or explicitly deferred (Section 11).

5.1 In scope for MVP

- 8 monitor types: HTTP(S), Keyword, Ping, Port, SSL certificate, Domain expiration, Cron/Heartbeat, DNS
- Best-effort multi-location checks (2–3 free-tier regions)
- Free-channel alerting: Email, Webhook, Telegram, Discord, Slack, Web push
- Incident detection, logging, comments, postmortem template, filtering, CSV export
- One public status page with branding, announcements, and email subscribers
- Per-monitor and fleet-wide analytics dashboard
- Small, manually curated dependency-monitoring list (15–20 major providers)
- Deployment-aware incident correlation (webhook-based, no AI)
- Root-cause timing breakdown (DNS/TCP/TLS/TTFB, no AI)

- Basic PDF/CSV report export
- Single-user account with API key access and a REST API

5.2 Out of scope for MVP

- SMS and voice-call alerts (require paid credits)
- The full 6,000+ provider dependency dataset (paid, licensed data)
- AI-assisted root-cause suggestions, natural-language monitor setup, AI-drafted status updates
- On-call scheduling and escalation policies
- Synthetic multi-step transaction monitoring
- Real user monitoring (RUM) / Core Web Vitals
- Multi-client agency workspaces
- Multiple user roles, seats, and any billing system

6. Functional Requirements

6.1 Core Monitoring

Each monitor type below follows the same lifecycle: create → scheduled check → result recorded → status evaluated → incident opened/closed on change.

Monitor type	Checks	Key config fields	Failure condition
HTTP(S)	Requests a URL, reads status/response	URL, method, interval, timeout, headers, expected status codes, keyword (optional)	Non-2xx/3xx status, timeout, or connection error
Keyword	Requests a URL, scans body for text	URL, keyword, exists/not-exists, case sensitivity	Keyword condition not met
Ping	ICMP echo to a host	Host/IP, interval, timeout, packet count	No response within timeout
Port	TCP connect to host:port	Host, port, interval, timeout	Connection refused or timeout
SSL certificate	Reads certificate expiry from a host	Host, warning threshold (days)	Days-to-expiry below threshold, or invalid chain
Domain expiration	Reads WHOIS/RDAP expiry date	Domain, warning threshold (days)	Days-to-expiry below threshold
Cron / Heartbeat	Waits for an inbound ping	Interval, grace period — generates a unique ping URL	No ping received within interval + grace period
DNS	Resolves configured records, diffs vs. last-known	Domain, record type(s) (A/AAAA/CNAME/MX/TXT/NS)	Unexpected record change

Multi-location checks

Best-effort for MVP: run checks from 2–3 free-tier compute regions (e.g. separate free-tier serverless functions/VPS instances in different regions) rather than a full global checker network. A failure only opens an incident after a second location confirms it — this is the primary false-positive defense and should not be skipped even at small scale.

Competitive note: UptimeRobot's own Free plan checks from a single default region only — multi-location is gated to paid plans there too (confirmed via the user's account screenshots, Section 6.11). This MVP's 2–3 region plan already matches or exceeds free-tier parity, so it is not a gap that needs closing before launch.

6.2 Alerts & Notifications

Channel	Setup	Notes
Email	SMTP or a free-tier transactional email API (e.g. Resend/Brevo free tier)	Default channel; always available

Channel	Setup	Notes
Webhook	User-provided URL, JSON payload	Enables custom automation for free
Telegram	Bot token + chat ID (free, user-created bot)	Popular free alternative to SMS
Discord	Incoming webhook URL	Free, common for indie/dev teams
Slack	Incoming webhook URL	Free tier sufficient for alerting
Web/mobile push	Browser Push API / PWA push	No third-party cost; requires the user to keep the tab/app installed

- Alert contacts are configured once, then attached to any monitor (not re-entered per monitor).
- Recurring/prolonging notifications: re-send on a configurable cadence while an incident stays open.
- Configurable notification delay (tolerate a slow endpoint before declaring down).
- Maintenance windows: scheduled downtime is excluded from alerting and from uptime calculations.

6.3 Incident Management

- Auto-detect: a failure opens an incident only after re-confirmation from a second location.
- Each incident records: monitor, start time, end time, duration, failure reason/detail.
- Comments: free-text notes on an incident; optionally shown on the public status page.
- Postmortem template: one click on a resolved incident creates a pre-filled retro doc (timeline, cause, resolution, follow-ups) for manual completion — no AI generation.
- Filter/sort by start date, end date, duration, status (ongoing/resolved).
- CSV export of incidents, per-monitor or account-wide.
- Exclude-from-reports toggle: keep a record without it affecting public uptime %.

6.4 Status Pages

- One public status page per account (MVP limit, matching the free-tier scope).
- Add any monitor as a component; group and order components.
- Branding: logo, accent color, page title.
- Announcements: post incident/maintenance updates with timestamps, shown in a timeline.
- Email subscribers: visitors opt in and get notified on status changes (free-tier email service).
- Per-component uptime bar (90-day, day-segmented) and response-time chart.
- Optional password protection for a private status page.

6.5 Analytics Dashboard

Per-monitor view

- Uptime % over 24h / 7d / 30d / 90d / custom range
- Response-time trend chart (avg, p50, p95)
- Incident count and total downtime for the selected range

- Mean time to recovery (MTTR)
- DNS / TCP / TLS / TTFB timing percentiles over time
- Full check-history table (paginated)

Fleet-wide (overview) view

- Aggregate uptime % across all monitors
- Fleet-wide response-time trend
- “Top offenders” — monitors responsible for the most downtime this period
- Incident volume trend over time
- % of incidents correlated with a recent deploy (see 6.7)

6.6 Dependency Monitoring (Small Free Version)

Rather than licensing a paid provider-incident dataset, the MVP tracks a short, manually curated list of ~15–20 major providers (e.g. AWS, Cloudflare, GitHub, Stripe, Google Cloud, Vercel) by polling each provider's own public status-page feed (most Statuspage.io- or similar-powered pages expose a free JSON/RSS endpoint). This has no licensing cost, at the cost of covering far fewer providers than a paid dataset would.

- User selects which tracked providers apply to them.
- Alerts route through the same free alert-contact system as regular monitors.
- Provider incidents are shown on their own timeline, separate from the user's own monitors (but see 6.7/6.8 for correlation).
- Does not count against the monitor quota.

6.7 Deployment-Aware Incidents

Free to build: this is pure data correlation, not AI. The system accepts inbound deploy webhooks (GitHub Actions, Vercel, Render, Railway, or a generic webhook) and timestamps each one. When an incident opens, the system checks whether a deploy for the related service happened within a configurable window (default 5 minutes) beforehand, and if so, surfaces it directly on the incident: “started 90 seconds after deploy #4821.”

6.8 Root-Cause Timing Breakdown

Also free: every HTTP(S) check already measures DNS lookup, TCP connect, TLS handshake, and time-to-first-byte as part of making the request. Storing and displaying that breakdown (and flagging which stage regressed most vs. the monitor's own 30-day baseline) requires no external service — it's a presentation of data the checker already has.

6.9 Reports & Export

- One-click PDF export of a status page's uptime/incident summary for a chosen date range (open-source PDF generation, e.g. a headless-browser or server-side PDF library — no paid API).
- CSV export of raw incident and check data.
- No scheduled/automated report delivery in MVP (manual export only) — scheduling can be added later without new cost.

6.10 Account, Auth & API

- Single user account per deployment; no team roles or seats in MVP.
- Email/password login (self-hosted) or a free OAuth provider (e.g. GitHub) — avoid paid auth services.
- One API key for programmatic access.
- REST API covering: monitor CRUD, incident read/comment, status-page CRUD, alert-contact CRUD.
- Deploy-webhook ingestion endpoint (see 6.7).

6.11 Onboarding & UX Patterns

Informed directly by the user's own UptimeRobot account (Free plan), screenshotted 2026-09-28 — see Appendix A.

Gamified onboarding checklist

A dismissible checklist widget shows on the near-empty dashboard, with a progress bar, guiding the user through: create your first monitor, set up an alert channel, connect a deploy webhook (this product's own differentiator, not UptimeRobot's), and create a status page. UptimeRobot ties its checklist to "25 free SMS credits" as the completion reward; since this product has no SMS tier to dangle, the reward is unlocking a short "what your dashboard can already tell you" tour instead.

Monitor creation: one scrollable form, not a wizard

UptimeRobot's own Add Monitor screen is a single long form with a sticky right-hand jump nav ("Monitor details / Integrations & Team / Maintenance info"), not a multi-page wizard. Section 9.1 adopts the same pattern: faster to build than a wizard, and lets the user see the whole shape of a monitor at once, with advanced settings tucked into collapsed sections.

Empty and success states

Match UptimeRobot's friendly, low-key tone for states with nothing to report (their incidents panel reads "■ Good job, no incidents. Keep it up!") rather than a blank table or a generic "No data."

Transparent-limits pattern (adapted)

UptimeRobot shows every plan-locked field inline with a note on what unlocks it, rather than hiding it. This product has no paid tier to upsell into during MVP, but the same transparency applies to *deferred* features (Section 11): where relevant, the UI can note "not in this version yet" rather than silently omitting a control a user might expect to find.

7. Non-Functional Requirements

Area	Requirement
Cost	\$0 ongoing spend across hosting, database, email, and all integrations for the scope in Section 5.1
Reliability	The monitoring system itself must not become a single point of failure — checks should keep running even if the dashboard UI is briefly unavailable
Check accuracy	A single failed check must never alone open an incident — always re-confirm from a second location/run first
Security	Passwords hashed (never stored in plaintext); API keys revocable; webhook endpoints validated (signature or shared secret) to prevent spoofed deploy/incident data
Data retention	At minimum 90 days of check history retained within free-tier storage limits; older data may be aggregated/rolled up rather than deleted outright
Portability	No proprietary lock-in to a single free-tier provider — architecture should be portable to a paid tier of the same or a different provider if usage grows
Accessibility	Status pages and dashboard should meet basic accessibility standards (readable contrast, semantic HTML, keyboard navigation)

8. Suggested Free-Tier Architecture

This section is guidance, not a mandate — the engineering approach is a separate decision, but the PRD notes free-tier options that are known to satisfy the \$0 constraint at small scale.

Layer	Free-tier option(s)	Notes
Checker workers	Scheduled serverless functions (e.g. Cloudflare Workers Cron Triggers, Vercel Cron, GitHub Actions scheduled workflows) in 2–3 regions	Free tiers have execution-frequency limits — design intervals around them (e.g. 5-minute minimum)
Database	Free tier of a managed Postgres (e.g. Supabase, Neon) or SQLite on a free-tier VM	Watch free-tier storage/row caps; plan for data rollup of old check history
Web app / API	Free tier of a platform like Vercel, Render, Railway, or Fly.io	Most have a free web-service tier suitable for a single-user app
Email sending	Free tier of a transactional email API (e.g. Resend, Brevo, Mailgun sandbox)	Free tiers typically cap monthly volume — fine for a single user's alert + subscriber volume
Status page hosting	Same platform as the web app, or a static export to a free static host	Keep load times low (NFR in Section 7)
PDF generation	Open-source library (e.g. a headless-browser print-to-PDF, or a server-side PDF library)	No paid API required

9. Key User Flows

9.1 Create a monitor

One scrollable form with a sticky section-jump nav, modeled directly on UptimeRobot's own Add Monitor screen (Section 6.11) — not a multi-page wizard.

- 1 User clicks "Add Monitor."
- 2 Picks a monitor type (HTTP, Keyword, Ping, Port, SSL, Domain, Cron, DNS), enters the target (URL/host/domain), and sets the interval — all on the same page.
- 3 Scrolling down (or jumping via the sticky nav) reaches notification channels — email is pre-filled from the account by default — then advanced settings (timeout, headers, keyword rule, grace period) inside a collapsed, expandable section.
- 4 Attaches one or more alert contacts (or creates a new one inline).
- 5 Saves — the monitor begins checking immediately, status shows as "pending" until the first check completes.

9.2 Incident lifecycle

- 1 Scheduled check fails.
- 2 System re-checks from a second location before treating it as real.
- 3 If confirmed, an incident opens and alerts fire immediately on all attached channels.
- 4 If a deploy happened in the correlation window, it's linked automatically (6.7).
- 5 Alerts recur on the configured cadence while the incident stays open.
- 6 Recovery is detected by the next successful check; the incident closes, alerts stop, and the event is logged with duration and reason.
- 7 User can add a comment, optionally push it to the status page, and start a postmortem.

9.3 Create a status page

- 1 User clicks "Create Status Page."
 - 2 Names the page and picks which monitors to show as components.
 - 3 Optionally sets branding (logo, color) and enables password protection.
 - 4 Publishes — the page is live at a generated URL.
-

10. Data Model Overview

High-level entities and their key relationships (not a full schema):

Entity	Key fields	Relationships
Monitor	id, type, target, interval, config (JSON), status	has many Checks, has many Incidents, belongs to many AlertContacts, appears on StatusPages
Check	id, monitor_id, timestamp, result, timing breakdown (dns/tcp/tls/ttfb)	belongs to Monitor

Entity	Key fields	Relationships
Incident	id, monitor_id, start_time, end_time, reason, excluded_from_reports	belongs to Monitor, has many Comments, may link to one Deploy
Comment	id, incident_id, body, visible_on_status_page	belongs to Incident
AlertContact	id, channel, config (JSON)	many-to-many with Monitor
StatusPage	id, name, branding, password_hash (optional)	has many Components (Monitor references)
Deploy	id, source, external_ref, timestamp	may be linked from Incident
Provider	id, name, status_feed_url	has many ProviderIncidents
ProviderIncident	id, provider_id, start_time, end_time, raw_update	belongs to Provider

11. Out of Scope for MVP (Deferred)

Deferred feature	Why it's deferred	Revisit when
SMS / voice-call alerts	Requires paid per-message/per-call credits (e.g. Twilio)	There's budget for a pay-as-you-go alerting add-on
Full 6,000+ provider dependency dataset	Licensed/paid data source	There's budget to license a dataset or build an equivalent
AI root-cause suggestions	Per-call LLM API cost	There's budget for API usage, or a free-tier model proves reliable enough
Natural-language monitor setup	Same as above	Same as above
AI-drafted status updates	Same as above	Same as above
On-call scheduling / escalation	High build complexity for a single-user product	There's a real multi-person on-call need
Synthetic multi-step transactions	Very high build complexity	Core single-check monitoring is stable and there's demand for it
Real user monitoring (RUM)	High complexity, needs client-side SDK + ingestion pipeline	Same as above
Multi-client agency workspaces	High complexity, needs multi-tenancy	There are real agency/multi-client users to design for
Multi-user roles, seats, billing	Out of scope while single-user and free	The product needs to support more than one person

12. Risks & Assumptions

Risks

- **Free-tier limits.** Hosting/database/email free tiers can change quotas or pricing with little notice — the architecture should stay portable (Section 8) so a provider swap doesn't require a rewrite.
- **Checker reliability from a single account.** Running checkers on free-tier serverless platforms may hit cold-start delays or scheduling jitter, affecting detection-time accuracy.
- **Provider status feeds change format.** The small dependency-monitoring list depends on each tracked provider's public feed staying stable and free.
- **Email deliverability.** Free-tier transactional email can land in spam more often than a paid, reputation-managed sender — monitor delivery success (Section 2.2) closely.

Assumptions

- The owner is comfortable self-hosting or using free-tier PaaS platforms rather than buying a managed monitoring SaaS.

- Monitor volume stays small (dozens, not thousands) for the life of this MVP — free tiers are not designed for large-scale fleets.
 - No SLA is being sold to paying customers yet, so free-tier reliability trade-offs are acceptable.
-

13. Roadmap

Phase	Focus	Representative features
MVP (this document)	Prove the free-tier product works for one user	Sections 6.1–6.10 in full
V2	Add paid-optional depth once there's a reason to charge	SMS/voice add-on, on-call scheduling, synthetic transactions, multi-user roles
Future	Scale and AI-assisted depth	AI root-cause suggestions, natural-language setup, RUM, multi-client agency workspaces, licensed dependency dataset

14. Open Questions

- 1 Which specific free-tier hosting/database/email providers should be locked in for the initial build?
 - 2 How many free-tier checker regions are realistically available/affordable at \$0 (2 vs. 3)?
 - 3 Should the status-page subscriber list be capped to stay within a free email tier's monthly send limit, and if so, at what number?
 - 4 Is a single status page per account acceptable long-term, or should that become a V2 priority sooner than other deferred items?
-

Appendix A — Competitive Research Basis

This PRD builds on a separate research document, “UptimeRobot Deep-Dive & Differentiation Strategy,” compiled from UptimeRobot's public marketing pages, their own live status page, their API v3 and MCP documentation, and their monitor-type/feature pages — not from the authenticated UptimeRobot dashboard, which was not accessed. That document contains the full feature audit, monitor-type comparison, incident-lifecycle diagram, and the complete differentiation strategy (unique features, 15 “wow” ideas, and MVP/V2/Future rationale) that this PRD's Section 5 and Section 11 scope decisions are drawn from.

Note on verification: feature claims about UptimeRobot in that source document are tagged Verified, Documented (not directly observed), Not accessible without authentication, or Suggested opportunity. This PRD's own feature set (Section 6) is original scope for this product, adapted specifically for a \$0-cost, single-user build — it is not a description of what UptimeRobot itself offers.

2026-09-28 update: the user logged into their own UptimeRobot account and shared screenshots of the authenticated Free-plan dashboard (monitor list, monitor detail page, and the full Add Monitor form). Those screenshots upgraded several previously “not accessible without authentication” items to directly verified, and corrected one earlier inference (the monitor-creation flow is a single scrollable form, not a multi-step wizard). The

findings are folded into Section 6.11 and the notes in Sections 6.1 and 9.1 above.